Lab 7: Proximity Search API

Roll No: 12340100
Name: Aditya Yadav

Abstract: This report describes a proximity search API for a 10,000-point grid. The API filters locations using circular distance and ranks the valid locations using actual grid or road traversal distance. In short, it checks who is near the house and then asks the roads how annoying the journey really is.

1. Objective

The objective was to recommend the ten closest locations for a given latitude, longitude, category, radius and optional road-linkage file. The required route is /IP/search/. The response is a JSON list of location IDs, with no decorative wrapper around it.

2. Working API

Working public URL: http://10.1.75.51:3204/IP/search/

Health check: http://10.1.75.51:3204/health

The API accepts GET query parameters and POST multipart form data. POST is recommended when a linkage file is uploaded.

Field	Type	Meaning
lat	float	Current latitude
long	float	Current longitude
cat	text	Requested category
rad	float	Circular eligibility radius
link	file, optional	Road links in text format

Example terminal request:

```
curl -X POST "http://10.1.75.51:3204/IP/search/" \
  -F "lat=0.5" -F "long=0.5" -F "cat=bank" -F "rad=0.8" \
  -F "link=@lab7/link.txt"
```

3. System Design

The CSV is loaded into memory when the service starts. Indexes are created for IDs, grid coordinates, latitude and longitude values, categories and spatial buckets. The regular grid uses four-neighbour links. Diagonal links are not used.

For each request, candidate points are first filtered by category and circular Euclidean radius. Dijkstra's algorithm then calculates the shortest traversal distance from the nearest grid node. The ten lowest distances are returned, with location ID used to make ties stable.

The supplied linkage file uses four values per line in this order:

```
Longitude_A Latitude_A Longitude_B Latitude_B
```

The sample file that was provided in google classroom contains 14,800 valid links. The parser converts these coordinates into graph edges and uses the resulting graph for ranking.

4. Deployment Details

Machine	Lab7 port	Role
sys1	8101	API worker
sys2	8102	API worker
sys3	8103	API worker
sys4	8104	API worker and public gateway

Nginx on sys4 exposes the public port 3204 and forwards /IP/search/ to the lab7 service on port 8104. The older JalaSetu terrain service ports were left alone.

5. Load and Resiliency Measures I took in this deployment

- Supervisor manages each lab7 process with autostart and autorestart enabled
- Each API node runs two Uvicorn workers
- Each worker has a 64-connection concurrency limit and a 2,048-connection backlog
- Process groups are stopped together so old workers do not remain hidden
- Non-finite coordinates and radii are rejected before graph work begin
- Large-radius bucket scans are clamped to the dataset boundary
- Linkage uploads larger than 64 MiB are rejected with HTTP 413
- Only two parsed custom graphs are kept in the cache
- Supervisor logs use rotation instead of growing forever like a very committed diary

6. Some Testing done and Observations

Test	Result
Randomized Dijkstra comparison	Passed
GET and POST contract	Passed
Zero radius	Passed
Large radius 999999	Passed
Malformed linkage	HTTP 400
NaN or infinite values	HTTP 422
20 concurrent clients, 200 requests	200 successful
Supervised process termination and recovery	Recovered

The final local regression suite passed 6 tests. The 20-client load test completed 200 requests successfully. Earlier measurements showed approximately 73–77 MB resident memory on sys4 during load.

7. Known Limitations

- The full dataset and indexes are held in memory, so startup cost grows with dataset size
- A request above the concurrency limit can be rejected(i don’t wanna cook my cluster) instead of being allowed to consume unlimited memory
- The public URL requires access to the relevant network or VPN
- If fewer than ten locations satisfy the category and radius conditions, fewer than ten IDs can be returned
- A malformed linkage file is rejected; the system does not silently replace it with a different road network

8. Conclusion

The API meets the required route and input contract, supports the coordinate-based linkage sample, and ranks candidates using road or grid traversal rather than only straight-line distance. The deployment has process supervision, bounded request work and recovery checks. It is not magic, but it is considerably less likely to panic when the evaluator arrives with ten hidden queries and a suspiciously large file.

*LibreOffice and my willpower was used to type out this report.

